Università degli Studi di Palermo

dipartimento di ingegneria unipa

Tipologie delle iterazioni

CALCOLATORI ELETTRONICI – FONDAMENTI DI PROGRAMMAZIONE

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

A che servono le iterazioni?

- I cicli vengono usati in particolari condizioni tipiche:
  - Cicli a conteggio → conosco a priori il numero di iterazioni
  - Cicli condizionali generici → l'iterazione continua finché una certa condizione non si avvera
  - Cicli sentinella → iterazioni di input di dati che continuano finché non si inserisce un *valore speciale* (valore *sentinella*)

A che servono le iterazioni?

- I cicli vengono usati in particolari condizioni tipiche:
  - Cicli controllati dalla fine di un file → iterazioni di input di valori da file che continuano finché non si raggiunge la fine (in C la costante `EOF`)
  - Cicli di validazione degli ingressi → l'iterazione continua finché l'input dell'utente non ricade in un certo range richiesto

Iterazioni a conteggio

```
/* Compute the payroll for a company */

#include <stdio.h>

int
main(void)
{
    double total_pay;      /* company payroll        */
    int    count_emp;      /* current employee       */
    int    number_emp;     /* number of employees    */
    double hours;          /* hours worked           */
    double rate;           /* hourly rate            */
    double pay;            /* pay for this period    */
```

Iterazioni a conteggio

```
/* Get number of employees. */
printf("Enter number of employees> ");
scanf("%d", &number_emp);

/* Compute each employee's pay and add it to the payroll. */
total_pay = 0.0;
count_emp = 0;
while (count_emp < number_emp) {
    printf("Hours> ");
    scanf("%lf", &hours);
    printf("Rate > $");
    scanf("%lf", &rate);
    pay = hours * rate;
    printf("Pay is $%6.2f\n\n", pay);
    total_pay = total_pay + pay;                /* Add next pay. */
    count_emp = count_emp + 1;
}
printf("All employees processed\n");
printf("Total payroll is $%8.2f\n", total_pay);

return (0);
}
```

Iterazioni a conteggio

- Operatori di assegnamento composti

- Servono ad abbreviare istruzioni del tipo: *var* = *var* <op> *exp*

```
+=
-=
*=
/=
%=
```

```
x = x * 4.56 → x *= 4.56
```

Iterazioni a conteggio

```
/* Process payroll for all employees */
total_pay = 0.0;
for (count_emp = 0;                          /* initialization              */
     count_emp < number_emp;                 /* loop repetition condition   */
     count_emp += 1) {                       /* update                      */
    printf("Hours> ");
    scanf("%lf", &hours);
    printf("Rate > $");
    scanf("%lf", &rate);
    pay = hours * rate;
    printf("Pay is $%6.2f\n\n", pay);
    total_pay = total_pay + pay;
}
printf("All employees processed\n");
printf("Total payroll is $%8.2f\n", total_pay);
```

Iterazioni a conteggio

- Operatori di auto-incremento e auto-decremento

- Semplificano ulteriormente gli incrementi/decrementi unitari delle variabili di conteggio

  ++

  --

- Sono operatori unari e possono essere in posizione *prefissa* o *postfissa*

Iterazioni a conteggio

- $++$*expr* prima incrementa *expr* di 1 e poi valuta il risultato
- $--$*expr* prima decrementa *expr* di 1 e poi valuta il risultato
- *expr*$++$ prima valuta *expr* e poi la incrementa di 1
- *expr*$--$ prima valuta *expr* e poi la decrementa di 1

Iterazioni a conteggio

Iterazioni a conteggio

Before...

i	j
2	?

Increments...

```
j = ++i;
```

prefix:
Increment `i` and then use it.

```
j = i++;
```

postfix:
Use `i` and then increment it.

After...

i	j
3	3

i	j
3	2

Iterazioni sentinella

```
/* Compute the sum of a list of exam scores. */

#include <stdio.h>

#define SENTINEL -99

int
main(void)
{
        int sum = 0,    /* output - sum of scores input so far          */
            score;      /* input - current score                        */

        /* Accumulate sum of all scores.                                 */
        printf("Enter first score (or %d to quit)> ", SENTINEL);
        scanf("%d", &score);      /* Get first score.                    */
        while (score != SENTINEL) {
            sum += score;
            printf("Enter next score (%d to quit)> ", SENTINEL);
            scanf("%d", &score);  /* Get next score.                 */
        }
        printf("\nSum of exam scores is %d\n", sum);

        return (0);
}
```

Si usa un valore convenzionale che non ricade nel range degli ingressi

È necessario acquisire un primo valore fuori dal ciclo

Iterazioni sentinella

```
/* Accumulate sum of all scores.                                    */
printf("Enter first score (or %d to quit)> ", SENTINEL);
for  (scanf("%d", &score);
      score != SENTINEL;
      scanf("%d", &score)) {
    sum += score;
    printf("Enter next score (%d to quit)> ", SENTINEL);
}
```

Iterazioni controllate dalla fine del file

```
#include <stdio.h>

int
main(void)
{
      int sum = 0,       /* sum of scores input so far */
          score,         /* current score  */
          input_status;  /* status value returned by scanf */

      printf("Scores\n");

      input_status = scanf("%d", &score);
      while (input_status != EOF) {
             printf("%5d\n", score);
             sum += score;
             input_status = scanf("%d", &score);
      }

      printf("\nSum of exam scores is %d\n", sum);

      return (0);
}
```

EOF è un valore sentinella

Iterazioni di validazione degli ingressi

```c
/*
 * Stampa il primo valore inserito all'interno dell'intervallo [N_MIN, N_MAX]
 */

#include <stdio.h>

#define N_MIN -3
#define N_MAX 7

int main(void){

        int in_val,     /* input - numero inserito dall'utente */
        status;                         /* valore di stao restituito da scanf */
        char skip_ch;                   /* carattere spurio da saltare */
        int error;      /* flag di errore per input errato */

        /* Acquisisci in_val dall'utente finché
                        non appartiene all'intervallo. */
        do {
                /* Nessun errore finora. */
                error = 0;

                /* Acquisisci il valore dall'utente. */
                printf("Inserisci un intero nel range da %d ", N_MIN);
                printf("a %d incluso> ", N_MAX);
```

Iterazioni di validazione degli ingressi

```
        status = scanf("%d", &in_val);

        /* Valida il valore del numero in ingresso. */
        if (status != 1) { /* non e' stato letto un numero */
            error = 1;
            scanf("%c", &skip_ch);
            printf("Carattere non valido >>%c>>. ", skip_ch);
            printf("Leggo il resto della linea.\n");
        }else if (in_val < N_MIN || in_val > N_MAX) {
        error = 1;
        printf("Il numero %d non è nel range.\n", in_val);

        /* Salta il resto della linea. */
        do
            scanf("%c", &skip_ch);
        while (skip_ch != '\n');
    }

}while (error);
printf("\nHai ilnserito il valore %d\n\n",in_val);
return 0;
}
```